https://traigent.ai/

Bazak Optimization Validation Report

Date	March 3, 2026
Experiment ID	`55a4bf49fd4730a3bb8e9bb4722f8dea`
Run ID	`06ab71ea-706d-4554-91df-40d6e0e552f6`
Optimization Method	Grid search, 4 model configurations, 3 evaluation examples each

1 Executive Summary

We completed an independent validation of the Bazak optimization results across all available system outputs and found no inconsistencies in the reported metrics.

Outcome

- Automated checks: **130**
- Passed: **130**
- Failed: **0**
- Overall conclusion: The reported results are internally consistent and can be shared.

2 What Was Validated

We compared the same run across four independent data outputs:

- Frontend CSV export (`configuration_runs_...csv`)
- SDK output (`examples/bazak/results.json`)
- SDK trial stream log (`trials_v2.jsonl`)
- SDK per-trial detail files (`trial_trial_*_v2.json`)

For each trial, we validated trial identity, model mapping, status, metric parity, latency/cost values, success-rate fields, example counts, and source-to-source deep consistency between JSONL logs and per-trial artifacts.

3 Data Evidence

3.1 Trial Set

Trial ID	Model
`trial_5822ccd7098e`	gemini-3-flash-preview
`trial_94093768acc4`	gemini-2.5-flash
`trial_4ce3058b9394`	gpt-5-nano-2025-08-07
`trial_444582cb961a`	gpt-5-mini-2025-08-07

All four sources contain the same trial IDs and model assignments.

3.2 Per-Trial Results (Consolidated)

Measure definitions used in this section

Raw quality measures

- `ExA` (Example Accuracy): Per-example accuracy for a single model+example pair. Formula per example i: ExA_i = mean of numeric per-example metrics whose names end with `_accuracy`. Important: this mean uses only keys returned for that example (sparse metrics), not a fixed 3-term denominator.
- `Tool Acc`: Trial-level correct tool selection rate.
- `Param Acc`: Trial-level correct tool-parameter rate.
- `Text Acc`: Trial-level text-response correctness rate.

Derived aggregate measures

- `Accuracy`: Trial-level mean of per-example accuracy:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N}\text{accuracy}_i, \quad N = 3$$

  Here, accuracy_i is the example-level `accuracy` returned for each evaluation example. Worked example (`gemini-2.5-flash`):

$$(1.0 + 1.0 + 0.5)/3 = 0.8333.$$

  In section 3.3, this value is shown as `Acc` and repeated for each row of the same trial. In this run it is **not** computed as

$$\frac{\texttt{tool_accuracy} + \texttt{param_accuracy} + \texttt{text_accuracy}}{3}.$$

  That split-metric mean is used only as a fallback when explicit `accuracy` is missing.

- `Score`: Same as `Accuracy` in this run. In section 3.3, shown as `Scr` and repeated per row of the same trial.

Operational measures

- `Lat(ms)`: Mean latency over examples in the trial (lower is better).
- `SDK$`: Total cost across all 3 examples (lower is better).
- `FE$`: Per-example cost displayed in FE (`SDK Total Cost / 3` for this run; lower is better).

Mdl	Raw			Drv		Ops		
	TAcc	PAcc	XAcc	Acc	Scr	Lat(ms)	SDK$	FE$
M1	0.67	1.00	1.00	0.83	0.83	11286	0.0247	0.0082
M2	0.67	0.00	1.00	0.67	0.67	16323	0.0537	0.0179
M3	0.33	0.00	1.00	0.50	0.50	36680	0.0197	0.0066
M4	0.33	0.00	1.00	0.33	0.33	32653	0.0048	0.0016

Mdl map: M1=gemini-2.5-flash, M2=gemini-3-flash-preview, M3=gpt-5-mini-2025-08-07, M4=gpt-5-nano-2025-08-07.

3.3 Raw Example-Level Accuracy and Trial Aggregation Check

Interpretation note

- `ExA` is per-example and can differ between E1/E2/E3 within the same trial.
- Trial-level values (`TA`, `PA`, `XA`, `Acc`, `Scr`, `Lat`, `SDK$`, `FE$`) are reported once per trial in section 3.2.
- `Acc` is validated as `Mean(ExA)` over the three examples.
- `TA`, `PA`, and `XA` are each averaged over examples where that specific key exists (missing keys are omitted, not treated as zero).

Trl	Model	ExA(E1)	ExA(E2)	ExA(E3)	Mean(ExA)	Trial Acc	Trial Scr
T1	gemini-2.5-flash	1.00	1.00	0.50	0.8333	0.8333	0.8333
T2	gemini-3-flash-preview	0.00	1.00	1.00	0.6667	0.6667	0.6667
T3	gpt-5-mini-2025-08-07	0.00	1.00	0.50	0.5000	0.5000	0.5000
T4	gpt-5-nano-2025-08-07	0.00	0.00	1.00	0.3333	0.3333	0.3333

Check equations

- T1: $(1.0 + 1.0 + 0.5)/3 = 0.8333$
- T2: $(0.0 + 1.0 + 1.0)/3 = 0.6667$
- T3: $(0.0 + 1.0 + 0.5)/3 = 0.5000$
- T4: $(0.0 + 0.0 + 1.0)/3 = 0.3333$

Example legend: E1=no-filter-single-search-trashcan-blue, E2=product-search-specific-model, E3=consultant-fridge.

3.4 Dedicated Metric: M1-Ex-Measure (Missing Accuracy Null Audit)

Based on raw trial artifacts (`trials/trial_trial_*_v2.json`), we added a dedicated audit metric for missing per-example accuracy values.

- **Metric name:** `M1-Ex-Measure`
- **Definition:** 1 if per-example `accuracy` is `null` or missing; otherwise 0

- **Scope:** each model (M1..M4) and each example (E1..E3)

Model map: M1=gemini-2.5-flash, M2=gemini-3-flash-preview, M3=gpt-5-mini-2025-08-07, M4=gpt-5-nano-2025-08-07.

Model	Example	Raw accuracy	M1-Ex-Measure
M1	E1	1.0	0
M1	E2	1.0	0
M1	E3	0.5	0
M2	E1	0.0	0
M2	E2	1.0	0
M2	E3	1.0	0
M3	E1	0.0	0
M3	E2	1.0	0
M3	E3	0.5	0
M4	E1	0.0	0
M4	E2	0.0	0
M4	E3	1.0	0

Result:

- Total per-example records checked: `12`
- Null/missing `accuracy` values found: `0`
- Null/missing rate: `0.0%`

Data artifacts included in the latest package:

- `data/m1_ex_measure_accuracy_nulls.csv`
- `data/m1_ex_measure_accuracy_nulls_summary.json`

3.5 Numeric Consistency Bounds

Across source-to-source comparisons, the largest absolute deltas were:

- Accuracy: `0.00003333`
- Score: `0.00003333`
- Tool accuracy: `0.00003333`
- Latency: `0.00003333 ms`
- Per-example cost: `0.00001100 USD`

These are expected formatting and rounding differences only.

4 Clarifications for Client Questions

4.1 Why FE "Cost" Differs from SDK "Total Cost"

- SDK trial metrics store total trial cost across all 3 examples.
- FE "Cost" is displayed as a per-example value in this run.
- Evidence (all trials): `FE Cost ~ SDK Total Cost / 3`.

This is a representation difference, not a data-integrity issue.

4.2 Why "Best Config" is gemini-3-flash-preview while gemini-2.5-flash has Higher Overall Accuracy

- Optimization objective for this run was **tool_accuracy** (single objective).
- **Important note:** `tool_accuracy` measures only whether the correct tool was selected. It does not directly reward parameter correctness (`param_accuracy`) or final text correctness (`text_accuracy`).
- gemini-3-flash-preview and gemini-2.5-flash tied on `tool_accuracy` (0.6667).
- Because this objective produced a tie, selection fell back to deterministic tie behavior for this run and picked gemini-3-flash-preview.
- If optimizing for overall `accuracy`, gemini-2.5-flash would rank highest (0.8333).

4.3 What If We Optimized for Other Accuracy Metrics?

Objective	Top Metric Value(s)	Selected Config	Practical Meaning
Tool accuracy	0.67 (tie)	gemini-3-flash-preview (tie handled)	Best tool-choice rate only
Param accuracy	1.00	gemini-2.5-flash	Best argument and parameter correctness
Text accuracy	1.00 (all 4 models)	No unique winner; tie across all models	Non-discriminative metric for this specific dataset
Overall accuracy	0.83	gemini-2.5-flash	Best end-to-end correctness under this scorer

Because all models scored `text_accuracy = 1.0000`, selecting this metric as the primary objective would not distinguish between configurations.

4.4 Content Novelty and Content Uniqueness

Displayed values in FE:

- Content Novelty: `0.5000`
- Content Uniqueness: `0.8336`

Meaning:

- **Content Uniqueness** estimates how different each evaluation input is from other inputs in the same dataset. Higher values mean less overlap and repetition.

- **Content Novelty** estimates how far an example is from the dataset center (how atypical it is relative to the rest).

Why these are the same across all four trials:

- They are dataset-level descriptors, not model-quality metrics.
- They are derived from the evaluation examples themselves and then shown consistently across model trials for that run.

Interpretation for this run:

- `0.8336` uniqueness indicates the three examples are fairly distinct from each other.
- `0.5000` novelty is a neutral value in this run and should not be used to compare model quality.

4.5 Service-Provider Metric Behavior (Client-Side Handling)

Live API spot-checks run on **March 3, 2026** against `POST /traigent/v1/evaluate` showed the following response behavior:

- Per-example metric keys are **sparse** (different examples may return different subsets of `tool_accuracy`, `param_accuracy`, `text_accuracy`).
- The evaluate response does **not** include explicit per-example `accuracy`.
- Aggregate metric keys can still include `text_accuracy` even when that key appears only on a subset of examples.

Client-side handling implemented in this run:

- Per-example `ExA` is derived as the mean of available numeric `*_accuracy` keys for that example.
- Trial-level `Acc` is the mean of per-example `ExA`.
- Trial-level `TA` / `PA` / `XA` are each averaged over examples where that specific key exists (missing keys are omitted, not treated as zero).

Important logging limitation for auditability:

- Trial artifacts currently persist per-example `accuracy`, but not the full per-example split metric map (`tool_accuracy` / `param_accuracy` / `text_accuracy` by example).
- This means exact per-example split-key reconstruction is not fully possible from stored trial files alone after the run.

Recommended follow-up:

- Persist full per-example metric dictionaries in trial logs to make post-run derivations and denominator choices fully auditable.

5 Additional Verification Notes

- We independently reviewed the end-to-end metric flow across SDK, backend persistence and analytics, and frontend extraction/CSV export.
- We validated values and structural integrity (key presence and deep metric parity between JSONL and per-trial detail files).

6 Final Assessment

This optimization run shows strong internal consistency across all recorded outputs.

Final conclusion: The surprising result pattern appears to be a real outcome of the configured objective and dataset behavior, not evidence of a metric-propagation bug in this run.